



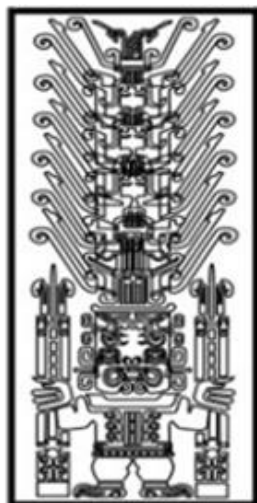
Universidad Nacional
Federico Villarreal



Facultad de Ingeniería
Industrial y de Sistemas

FACULTAD DE INGENIERÍA INDUSTRIAL Y DE SISTEMAS

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



**“Sistema inteligente de protección y control del secado de café basado
en condiciones climáticas”**

EQUIPO 5 - ENTREGABLE II

AUTOR

Franco Arias, Mauricio

Olivera Torres, Leonardo

Orellana Fernández, Alexandra

Palacios Hinostroza, Lesli

DOCENTE

Mg. Hernan Villafuerte Barreto

CURSO

Sistemas Digitales y Arquitectura de Computadoras

LIMA-LIMA-PERÚ

2026

ENTREGABLE 2

1. Diagrama de flujo del sistema :

El diagrama de flujo representa la secuencia de decisiones y acciones que realiza el sistema automatizado de control de cobertura basado en Arduino. A través de este se describe el funcionamiento lógico del programa, desde la lectura de los sensores de temperatura y lluvia, hasta la activación del servomotor que abre o cierra la estructura. Además, permite visualizar de manera ordenada cómo el sistema prioriza la detección de lluvia sobre la temperatura ya que es la condición de seguridad más crítica.

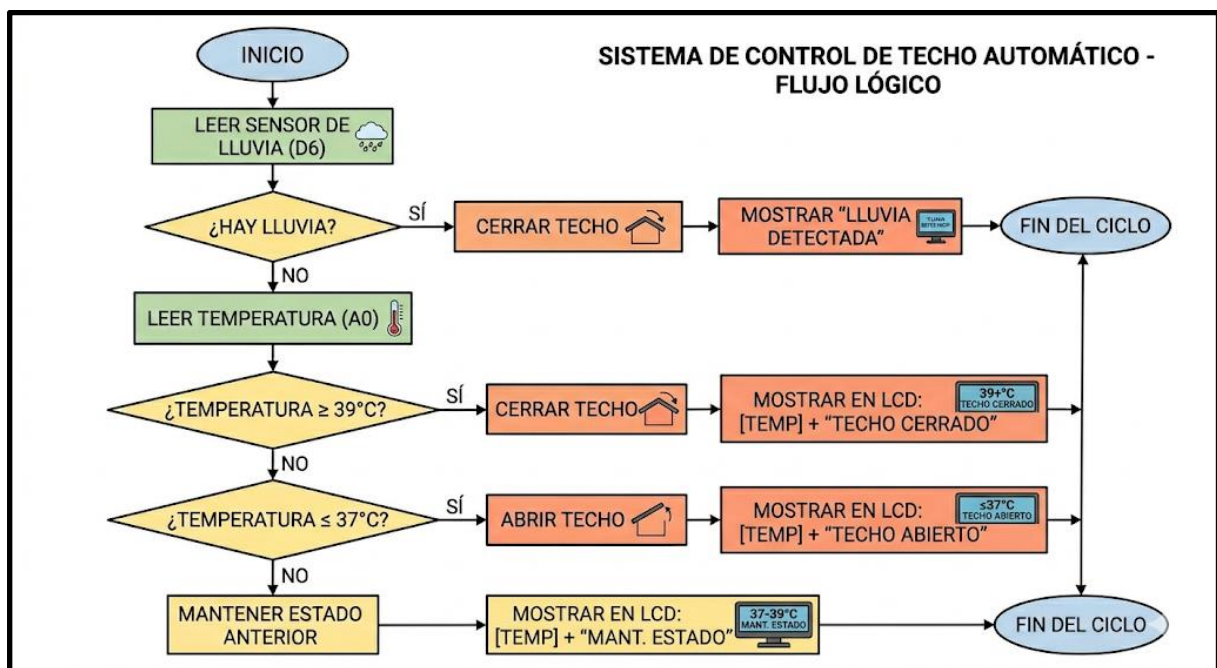


Diagrama de flujo

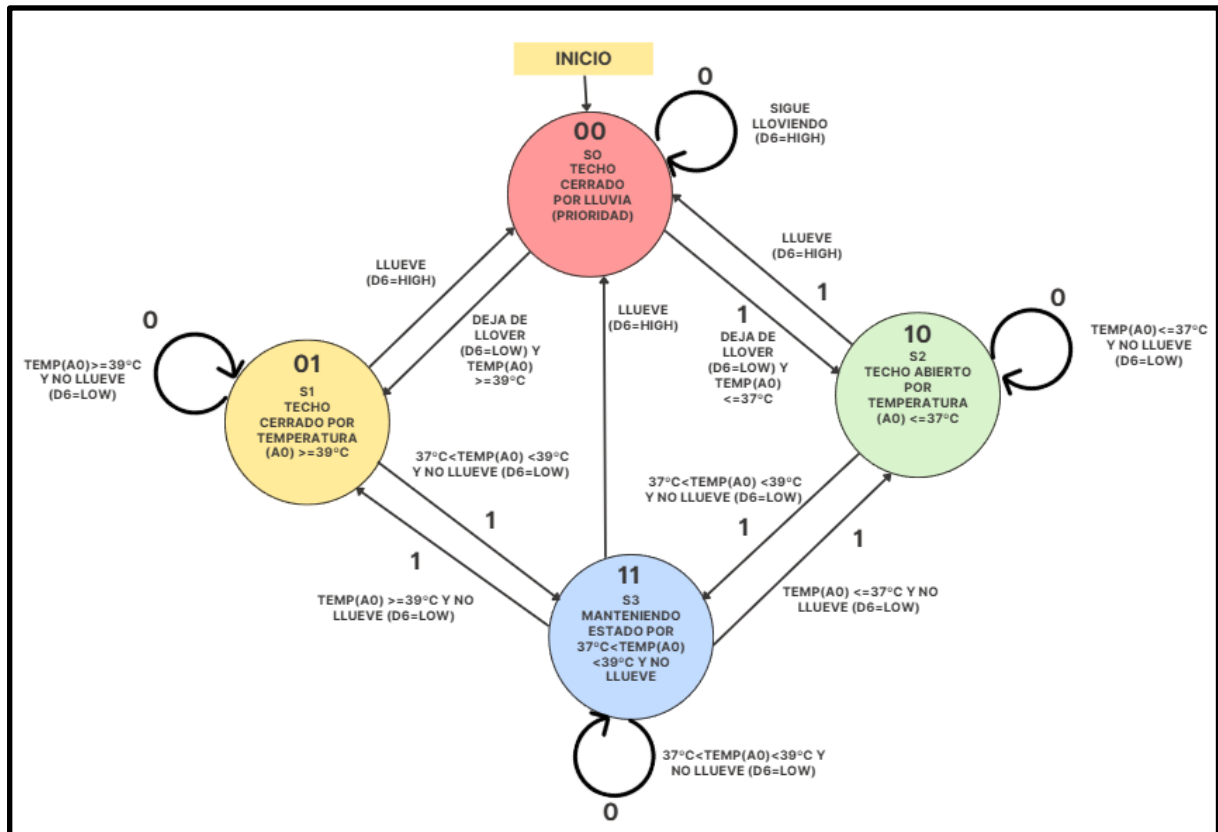
2. Máquina de Estados Finita (FSM):

La Máquina de Estados Finita (FSM) del sistema define los distintos estados en los que puede encontrarse el sistema automatizado de control de cobertura, así como las transiciones entre ellos según las condiciones de los sensores de temperatura y lluvia. El sistema solo puede estar en un número limitado de estados definidos, cambiando entre ellos de forma controlada.

- **Techo abierto:** Se activa cuando la temperatura es $\leq 37^{\circ}\text{C}$.
- **Techo cerrado por temperatura:** Se activa cuando la temperatura es $\geq 39^{\circ}\text{C}$, el sistema cierra el techo para proteger los granos de café de altas temperaturas.
- **Techo cerrado por lluvia (prioridad):** Se activa cuando el sensor de lluvia detecta agua, el sistema ignora la temperatura y el techo se cierra inmediatamente.

- **Manteniendo estado:** Se activa cuando la temperatura está entre 37°C y 39°C, el sistema no cambia la posición del techo y mantiene el último estado activo.

El sistema incluye un estado de “mantenimiento”, en el cual no se realiza ninguna acción sobre el servomotor, ya que las condiciones ambientales se encuentran dentro de un rango estable. Este estado permite evitar cambios innecesarios en el sistema.



Máquina de Estados Finita

0 = permanece

1= cambia

ESTADOS	Q1	Q0	C	A
S0	0	0	1	0
S1	0	1	1	0
S2	1	0	0	1
S3	1	1	0	0

Tabla de salidas

c = cerrado

a = abierto

3. Arquitectura del ATmega328P:

Es el chip principal del Arduino Uno, que lee sensores, procesa datos, ejecuta el programa y activa salidas (servo, LCD, etc.). Es de 8 bits (cantidad de datos que el microcontrolador puede procesar a la vez), procesa datos pequeños (en bloques de 8 bits), tiene memoria separada para programa y datos (más rápido).

Está compuesto por las siguientes partes:

- CPU (Unidad Central de Procesamiento)

Es el “cerebro” del chip, hace todo el procesamiento, lee el programa, toma decisiones y controla sensores y salidas

- Memoria Flash

Es la memoria permanente donde se guarda tu programa, el código del Arduino no se borra al apagar.

- SRAM

Es la memoria temporal, guarda cosas mientras el programa está funcionando como:

Variables, datos de sensores (DHT22, lluvia), estado del sistema y se borra al apagar.

- EEPROM

Memoria permanente para datos pequeños como: última temperatura registrada, último estado del servo (abierto o cerrado), etc. Guardar esa información aunque apagues el Arduino.

- ADC (Convertidor Analógico-Digital)

Convierte señales “reales” (analógicas) en números que el Arduino entiende, como DHT22 o sensores de temperatura.

- UART (Comunicación serial)

Permite la comunicación entre dispositivos, como ver el monitor serial de la computadora

- Timers (Temporizadores)

Sirven para medir tiempo internamente.

4. Mapa de memoria

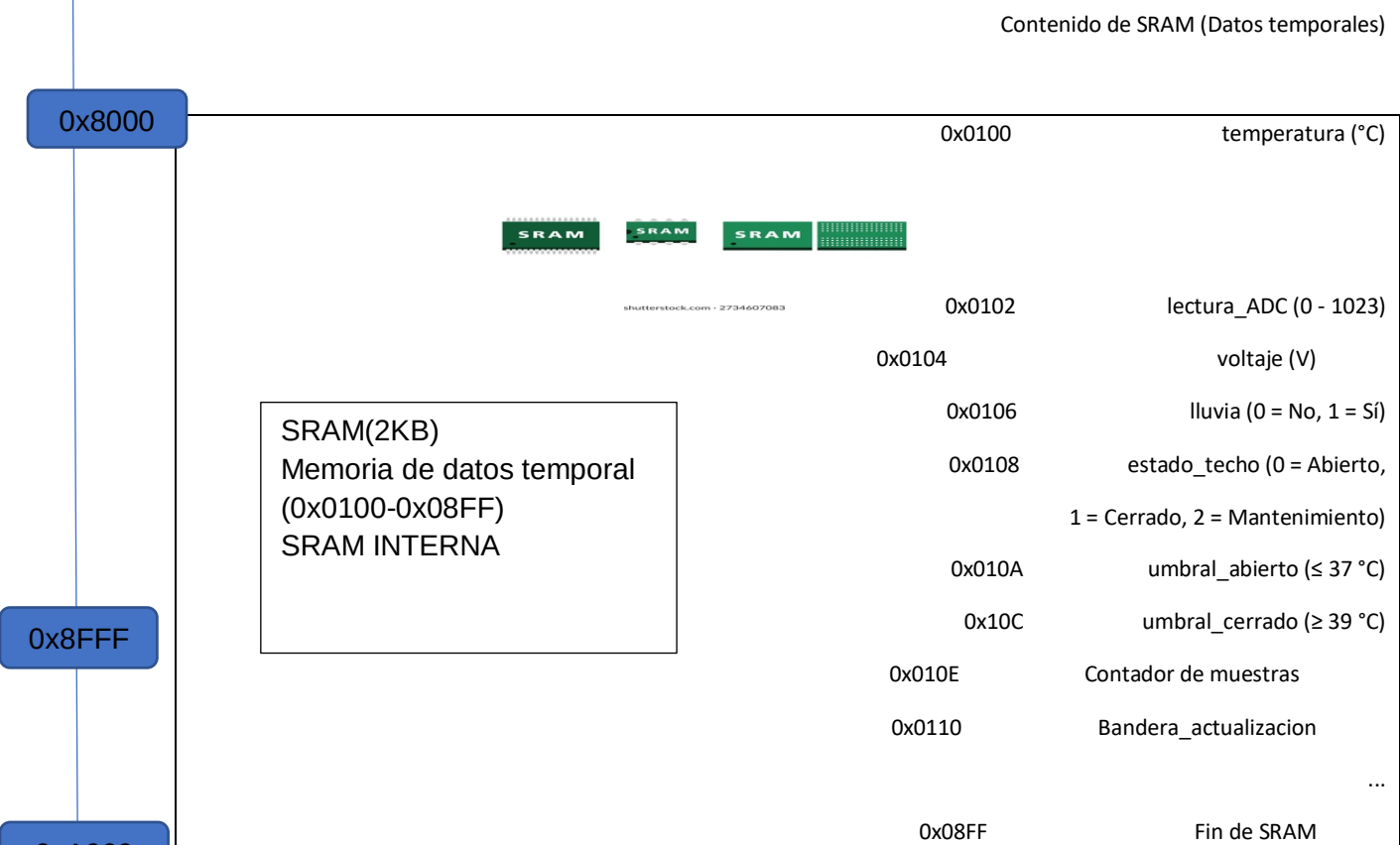
El mapa de memoria del microcontrolador ATmega328P permite visualizar la organización interna de los datos y del programa dentro del sistema. Esta arquitectura es fundamental para el correcto funcionamiento del sistema automatizado de control de cobertura del secado de café.

La memoria Flash es utilizada para almacenar el programa principal del sistema, incluyendo las instrucciones necesarias para la lectura de los sensores, el procesamiento de datos y el control del servomotor. Esta memoria es de tipo no volátil, lo que significa que conserva la información incluso cuando el sistema se encuentra apagado.

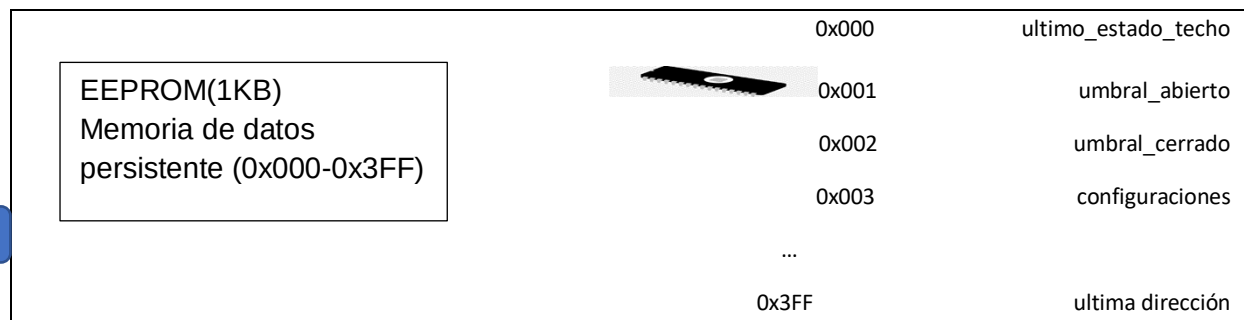
Por otro lado, la memoria SRAM se emplea para el almacenamiento temporal de los datos durante la ejecución del programa. En este sistema, se almacenan variables como la temperatura, la humedad ambiental, la detección de lluvia y el estado actual del techo. Esta memoria es volátil, por lo que su contenido se pierde al apagar el dispositivo.

Asimismo, la memoria EEPROM se utiliza para guardar datos persistentes que deben mantenerse incluso después de reiniciar el sistema, como el último estado del techo o configuraciones del sistema. Esto permite mantener continuidad en el funcionamiento del sistema ante interrupciones de energía.

De esta manera, la correcta distribución de la información en cada tipo de memoria permite optimizar el rendimiento del sistema, asegurando una respuesta eficiente y confiable ante cambios en las condiciones climáticas durante el proceso de secado del café.



Contenido de EEPROM



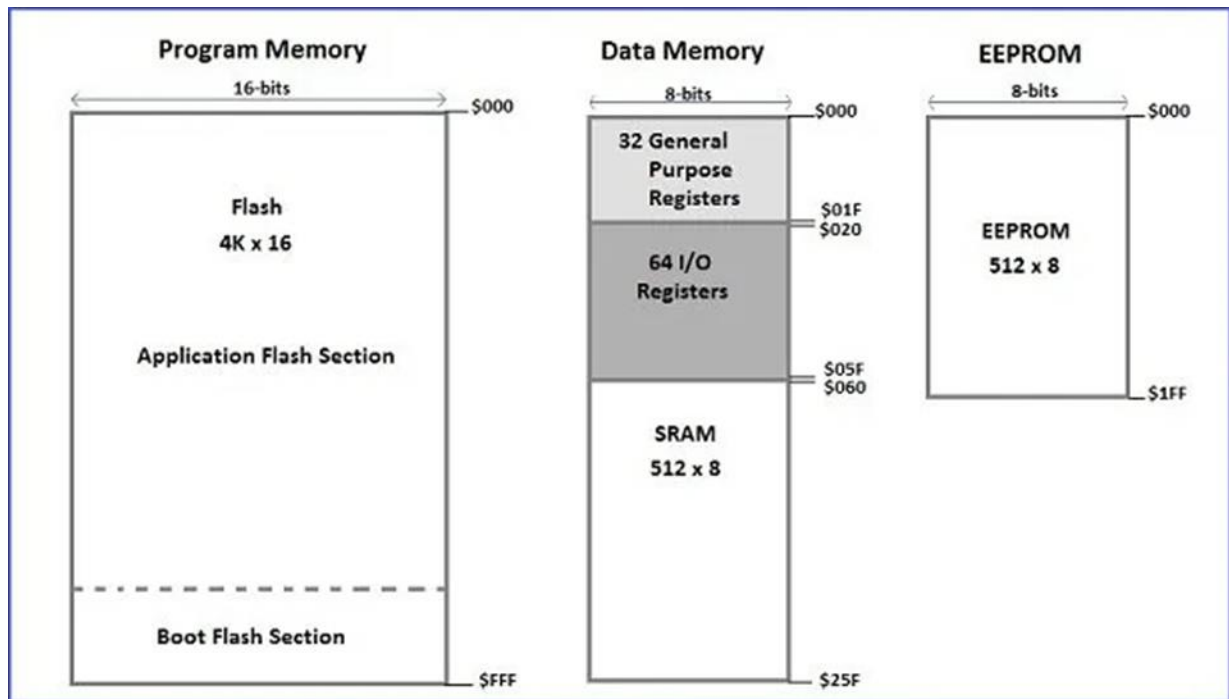
AMPLIANDO MEMORIA SRAM

REGISTROS I/O

DIRECCION	TAMAÑO	DESCRIPCION/USO
0x000-0x001F	32 bytes	REGISTROS CPU Registros internos del núcleo AVR(estado,control,punteros,etc.)
0x0020-0x005F	64 bytes	REGISTROS I/O Registros de entrada/salida para periféricos internos(puertos,timers,ADC,SPI,UART,etc.)
0x0060 – 0x00FF	160 bytes	I/O EXTENDIDO Registros de I/O extendidos.

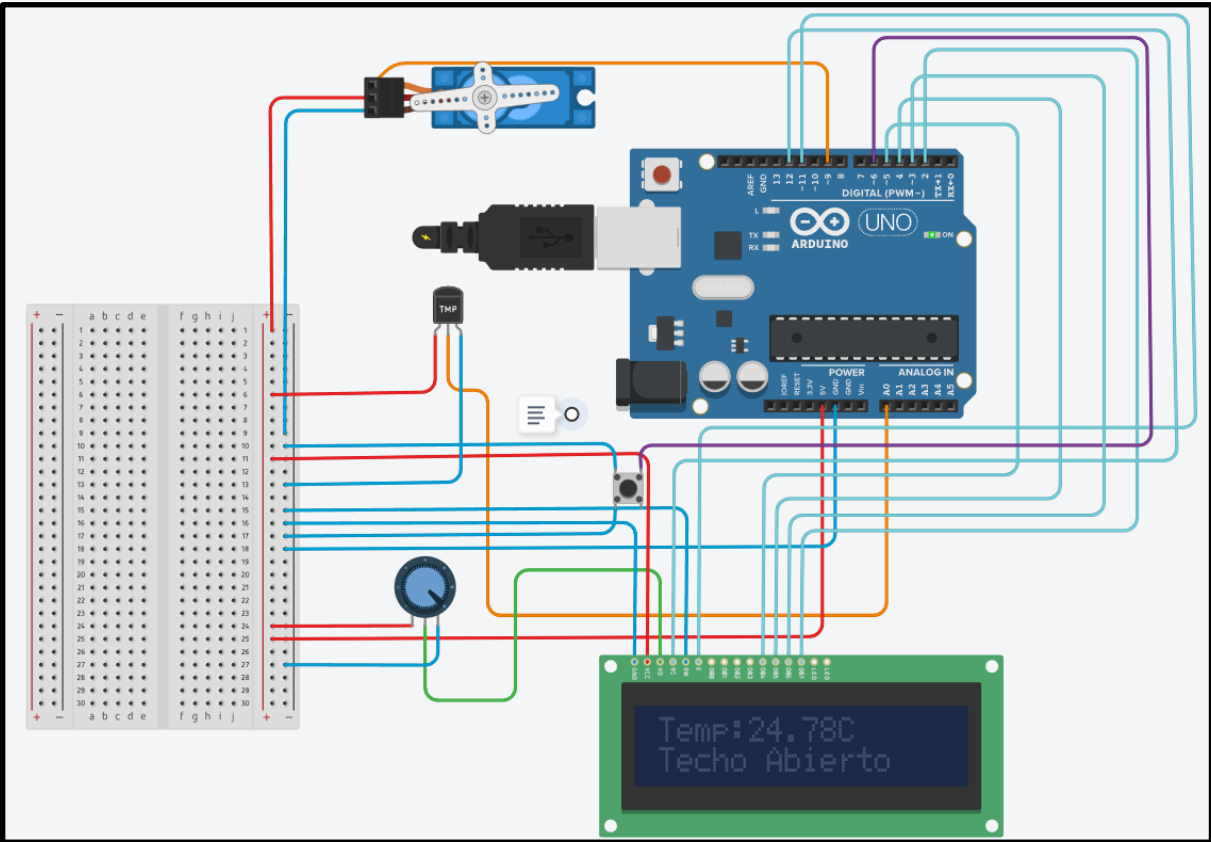
REGISTROS I/O MAS UTILIZADOS (direcciones en hexadecimal)

DIRECCION	REGISTRO	FUNCION/USO EN EL PROYECTO
0x0023	DDRB	Dirección de pines del puerto B (salidas PWM:servo)
0x0026	PINC	Lectura de pines del puerto C (ADC5: A5)
0x0027	DDRC	Direccion de pines del puerto C (entradas ADC)
0x0028	PORTC	Salida de pines del puerto C (activación pull-up)
0x0078	ADCL	ADC- Registro bajo (lectura del resultado)
0x0079	ADCH	ADC – Registro alto (lectura del resultado)
0x008B	TCCR1A	Control Timer/Counter 1 A (PWM para servo)
0x008C	TCCR1B	Control Timer/Counter 1 B (prescaler, modo)



5. Mapa de pines:

El mapa de pines permite mostrar de manera clara y estructurada la distribución exacta de las conexiones entre el microcontrolador (Arduino Uno) y los componentes periféricos. Esta tabla funciona como una guía técnica indispensable para garantizar que las señales del sensor de lluvia, el DHT22, el servomotor y la pantalla LCD viajen por los canales correctos, evitando conflictos de hardware y sirviendo como referencia directa para la programación del código.



Esquema Visual (Diagrama)

Componente	Pin del Componente	Conexión en Arduino / Protoboard	Función principal
Sensor de Lluvia	Señal (DO) / Botón	Pin Digital 6 (D6)	Detecta la presencia de lluvia.
Sensor Temp. (DHT22)	Datos (OUT)	Pin Digital 8 (D8)	Envía los datos de temperatura.
Servomotor	Señal (Naranja/Amarillo)	Pin Digital 9 (D9)	Controla la apertura/cierre del techo.
Pantalla LCD 16x2	RS, E, DB4, DB5, DB6, DB7	D12, D11, D5, D4, D3, D2	Muestra el estado del sistema.
LCD (Contraste)	V0	Centro del potenciómetro	Ajusta la claridad de los caracteres.
Alimentación	VCC / 5V	Pin 5V (Protoboard)	Suministra energía a los módulos.
Tierra	GND	Pin GND (Protoboard)	Cierra el circuito de todos los componentes.

Tabla de componentes y conexiones

6. Interfaces de comunicación

Las interfaces de comunicación permiten la conexión entre el Arduino Uno y los diferentes componentes del sistema. A través de ellas, el microcontrolador recibe datos de los sensores, procesa la información y envía señales de salida hacia el LCD, el servomotor y el monitor serial.

Componente	Tipo de interfaz	Pin usado	Comunicación
DHT22	Entrada digital de datos	D8	Envía la temperatura al Arduino.
Sensor de lluvia	Entrada digital	D6	Indica si hay lluvia o no.
LCD 16x2	Salida paralela de 4 bits	D12, D11, D5, D4, D3, D2	Muestra temperatura y estado del techo.
Servo	Salida de control por pulsos	D9	Recibe la orden para abrir o cerrar el techo.
UART	Comunicación serial	USB / TX-RX	Envía datos al Monitor Serial de la computadora.

Tabla de interfaces de comunicación del sistema

De esta manera, el sistema puede monitorear las condiciones climáticas y responder automáticamente abriendo o cerrando el techo según la temperatura o la presencia de lluvia.

7. Código comentado

El siguiente código corresponde al programa del sistema inteligente de protección y control del secado de café. Este programa permite leer las condiciones climáticas mediante los sensores de temperatura y lluvia, procesar la información en el Arduino Uno y controlar automáticamente el servomotor encargado de abrir o cerrar el techo. Además, muestra el estado del sistema en la pantalla LCD y en el Monitor Serial para facilitar su supervisión.

```
// Se incluye la librería LiquidCrystal para controlar la pantalla LCD 16x2
#include <LiquidCrystal.h>
```

```
// Se incluye la librería Servo para controlar el servomotor del techo
#include <Servo.h>
```

```
// Configuración de los pines conectados al LCD
// RS = 12, E = 11, D4 = 5, D5 = 4, D6 = 3, D7 = 2
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
```

```

// Se crea el objeto "techo" para controlar el servomotor
Servo techo;

// Pin digital donde se conecta el botón o sensor de lluvia
const int botonLluvia = 6;

void setup() {

    // Inicia la comunicación serial a 9600 baudios
    // Permite visualizar datos en el Monitor Serial
    Serial.begin(9600);

    // Inicializa la pantalla LCD de 16 columnas y 2 filas
    lcd.begin(16, 2);

    // Conecta el servomotor al pin digital 9
    techo.attach(9);

    // Configura el pin del sensor de lluvia como entrada con resistencia interna pull-up
    // Con INPUT_PULLUP, el valor LOW representa lluvia detectada
    pinMode(botonLluvia, INPUT_PULLUP);

    // Muestra mensaje inicial en la primera fila del LCD
    lcd.setCursor(0, 0);
    lcd.print("Sistema Cafe");

    // Muestra mensaje inicial en la segunda fila del LCD
    lcd.setCursor(0, 1);
    lcd.print("Iniciando...");

    // Espera 2 segundos antes de iniciar el funcionamiento del sistema
    delay(2000);

    // Limpia la pantalla LCD
    lcd.clear();
}

void loop() {

    // =====
    // LECTURA DEL SENSOR DE LLUVIA
    // =====

    // Lee el estado del sensor o botón de lluvia
    // LOW significa que se detectó lluvia
    // HIGH significa que no se detectó lluvia
    int lluvia = digitalRead(botonLluvia);

    // =====
    // LECTURA DE TEMPERATURA SIMULADA

```

```

// =====

// Lee el valor analógico del pin A0
// En la simulación, este pin representa la lectura de temperatura
int lectura = analogRead(A0);

// Convierte la lectura analógica a voltaje
// El Arduino trabaja con valores de 0 a 1023 equivalentes a 0V - 5V
float voltaje = lectura * 5.0 / 1023.0;

// Convierte el voltaje a temperatura en grados Celsius
// Esta fórmula corresponde a un sensor analógico tipo TMP36 usado en simulación
float temperatura = (voltaje - 0.5) * 100.0;

// =====
// ENVÍO DE DATOS AL MONITOR SERIAL
// =====

// Muestra la temperatura medida en el Monitor Serial
Serial.print("Temp: ");
Serial.print(temperatura);
Serial.print(" C | Lluvia: ");
Serial.println(lluvia);

// =====
// PRIORIDAD DEL SISTEMA: DETECCIÓN DE LLUVIA
// =====

// Si se detecta lluvia, el sistema cierra el techo inmediatamente
// Esta condición tiene prioridad sobre la temperatura
if (lluvia == LOW) {

    // Mueve el servomotor a 180 grados para cerrar el techo
    techo.write(180);

    // Muestra alerta de lluvia en la primera fila del LCD
    lcd.setCursor(0, 0);
    lcd.print("LLUVIA DETECTADA ");

    // Muestra el estado del techo en la segunda fila del LCD
    lcd.setCursor(0, 1);
    lcd.print("Techo Cerrado  ");

    // Envía mensaje de estado al Monitor Serial
    Serial.println("LLUVIA -> TECHO CERRADO");
}

// Si no hay lluvia, el sistema evalúa la temperatura
else {

```

```

// =====
// CONTROL POR TEMPERATURA ALTA
// =====

// Si la temperatura es mayor o igual a 39 °C,
// el sistema cierra el techo para proteger el café
if (temperatura >= 39) {

    // Cierra el techo moviendo el servomotor a 180 grados
    techo.write(180);

    // Muestra la temperatura en la primera fila del LCD
    lcd.setCursor(0, 0);
    lcd.print("Temp:");
    lcd.print(temperatura);
    lcd.print("C ");

    // Muestra el estado del techo en la segunda fila
    lcd.setCursor(0, 1);
    lcd.print("Techo Cerrado ");

    // Envía mensaje de estado al Monitor Serial
    Serial.println("TECHO CERRADO");
}

// =====
// CONTROL POR TEMPERATURA BAJA O NORMAL
// =====

// Si la temperatura es menor o igual a 37 °C,
// el sistema abre el techo para continuar con el secado
else if (temperatura <= 37) {

    // Abre el techo moviendo el servomotor a 0 grados
    techo.write(0);

    // Muestra la temperatura en la primera fila del LCD
    lcd.setCursor(0, 0);
    lcd.print("Temp:");
    lcd.print(temperatura);
    lcd.print("C ");

    // Muestra el estado del techo en la segunda fila
    lcd.setCursor(0, 1);
    lcd.print("Techo Abierto ");

    // Envía mensaje de estado al Monitor Serial
    Serial.println("TECHO ABIERTO");
}

```

```

// =====
// MANTENIMIENTO DE ESTADO
// =====

// Si la temperatura está entre 37 °C y 39 °C,
// el sistema no cambia la posición del techo
else {

    // Muestra la temperatura en la primera fila del LCD
    lcd.setCursor(0, 0);
    lcd.print("Temp:");
    lcd.print(temperatura);
    lcd.print("C  ");

    // Indica que el sistema mantiene el estado actual
    lcd.setCursor(0, 1);
    lcd.print("Mant. Estado  ");

    // Envía mensaje al Monitor Serial
    Serial.println("MANTENIENDO ESTADO");
}
}

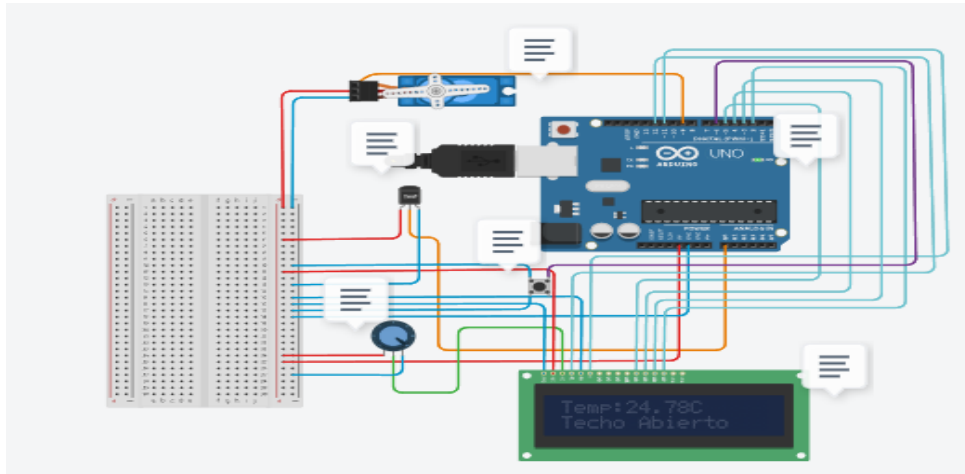
// Espera 1 segundo antes de realizar una nueva lectura
delay(1000);
}

```

8. Capturas de Tinkercad

Temperatura Baja o Ideal (Apertura por clima fresco)

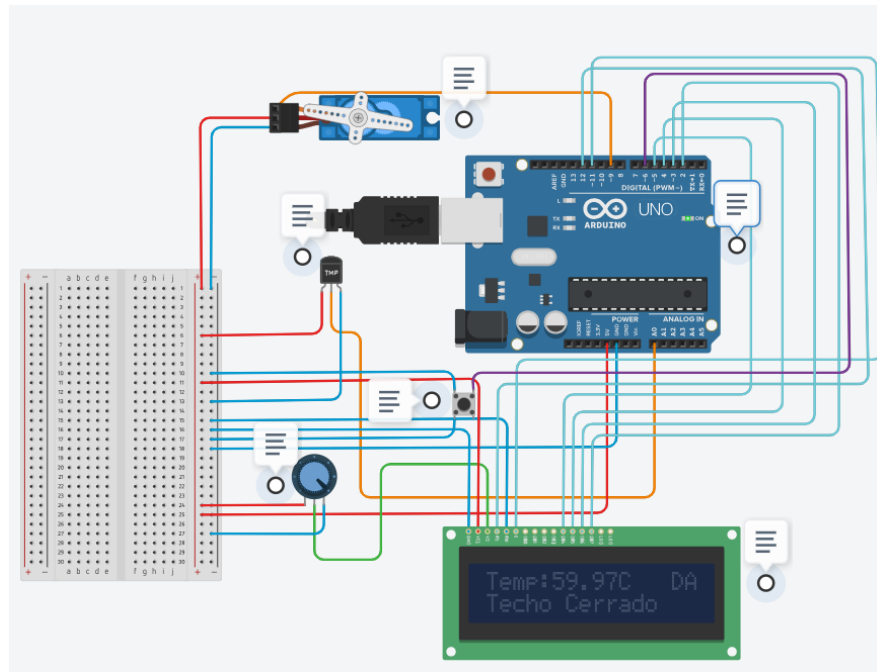
- El sol baja o el ambiente se enfría. El sensor TMP36 detecta que la temperatura ha descendido.
- El Arduino registra este cambio y le ordena al servomotor que cambie de posición.
- La pantalla LCD cambia su mensaje para mostrar **Temp: 24.78°C - TECHO ABIERTO.**



Exceso de Calor (Cierre por alta temperatura)

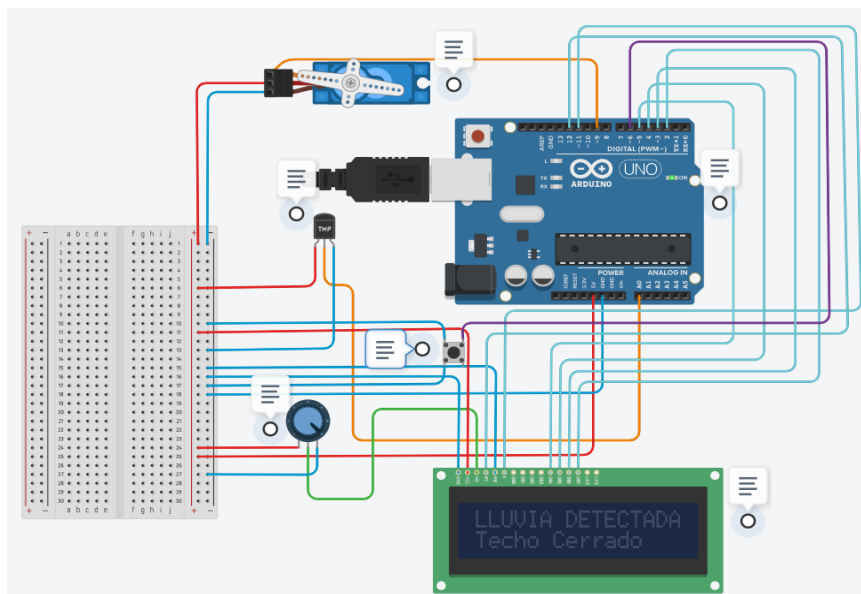
- El sensor de temperatura TMP36 detecta que el calor en el ambiente ha superado el límite
- El Arduino envía una orden eléctrica al servomotor que gire automáticamente hacia la posición de "cierre".

- La pantalla LCD mostrará un mensaje indicando el valor alto **Temp: 59.97°C - TECHO CERRADO.**



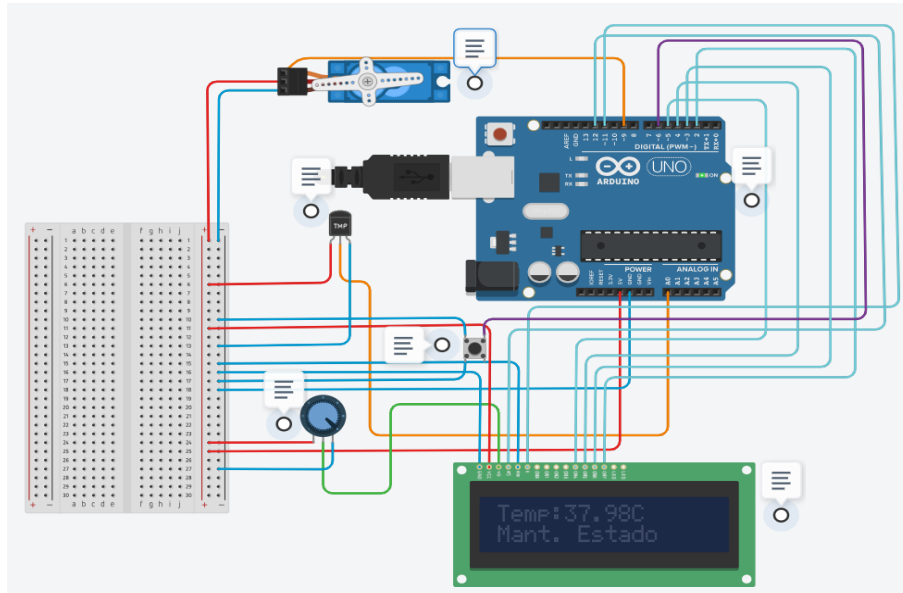
Simulación de Lluvia (Prioridad del botón)

- Presionas el pulsador y el sistema detecta "lluvia" y, por programación, ignora la temperatura.
- El servomotor gira inmediatamente a la posición de cierre para bloquear el agua.
- La pantalla LCD deja de mostrar la temperatura y muestra un mensaje como: **Lluvia detectada / TECHO CERRADO.**



Zona de Calor Extremo (Prevención)

- El sol sigue calentando y el sensor de temperatura TMP36 detecta que el ambiente ha entrado en un rango muy específico entre los 37°C y 39°C.
- El Arduino evalúa esta condición especial y no le envía una orden distinta al **servomotor**
- La **pantalla LCD** actualiza su información para alertarte de que el sistema aparecerá **Mantenimiento de estado**



9. Manejo de datos a nivel de registros/buffers

El sistema de protección de secado de café funciona gracias al microcontrolador ATmega328P. Aunque el código de Arduino utiliza comandos sencillos para leer sensores y mover motores, internamente el "cerebro" del sistema maneja toda esta información utilizando registros en el hardware y buffers en su memoria temporal SRAM



9.1. Registros para el control de Entradas y Salidas

Los registros son como pequeños "interruptores" internos en la memoria del microcontrolador que le indican cómo debe comportarse físicamente el hardware.

- Configuración de pines: Cuando en el código asignamos el sensor de lluvia al pin digital D6 y el servomotor al pin D9, internamente el Arduino modifica un registro de dirección para establecer que el D6 es un canal de entrada (para escuchar al sensor) y el D9 es un canal de salida (para enviarle órdenes al motor).
- Lectura de datos lógicos: Al ejecutarse la función `digitalRead()` para detectar si hay agua, el procesador no "mira" el pin físico, sino que lee un registro interno de datos que cambia instantáneamente a estado LOW o HIGH dependiendo de lo que detecte el sensor de lluvia.

9.2 Registros en la Conversión Analógica-Digital (ADC)

El procesador solo entiende de ceros y unos, por lo que no puede comprender "grados de temperatura" directamente. Para procesar la señal del sensor (conectado al pin analógico A0), se utiliza el ADC (Convertidor Analógico-Digital).

- La señal eléctrica ingresa y el hardware del ADC la evalúa, guardando el resultado crudo (un número entre 0 y 1023) en registros de conversión internos.
- Posteriormente, nuestro programa toma este dato de los registros y utiliza fórmulas matemáticas para convertir ese valor a voltaje y, finalmente, a la temperatura real en grados Celsius.

9.3 Uso de Buffers (Memoria temporal)

Un buffer funciona como una "sala de espera" temporal dentro de la memoria SRAM. Es vital para que el sistema fluya rápidamente y no se trabe perdiendo eventos importantes, como el inicio de una lluvia.

- Buffer de Comunicación Serial (UART): Enviar texto a la computadora (como `Serial.print("Temp: ")`) es un proceso lento para el microcontrolador. Por ello, el Arduino guarda primero todo el mensaje en un buffer. Mientras un sistema de hardware se encarga de enviar ese mensaje letra por letra por el cable USB, la CPU queda libre para seguir ejecutando el programa y vigilando el techo.
- Variables como buffers en la SRAM: En el código, las variables locales como `lectura`, `voltaje` y `temperatura` actúan como pequeños buffers dentro de la memoria SRAM. Estas retienen momentáneamente la información ambiental necesaria para que la Máquina de Estados decida si abre o cierra el techo, y luego se borran o actualizan en el siguiente ciclo de funcionamiento.

10. Explicación de la máquina de programa almacenado

El sistema implementado funciona bajo el principio de una máquina de programa almacenado, en la cual el microcontrolador ejecuta de manera continua un conjunto de instrucciones previamente almacenadas en la memoria Flash. Este enfoque permite que el sistema procese información del entorno y tome decisiones en tiempo real.

El funcionamiento del sistema se basa en el ciclo de ejecución de instrucciones, compuesto por tres etapas principales: obtención (fetch), decodificación (decode) y ejecución (execute).

En la etapa de obtención, el microcontrolador lee las instrucciones almacenadas en la memoria Flash, tales como la lectura de los sensores de temperatura, humedad (DHT22) y el sensor de lluvia.

Posteriormente, en la etapa de decodificación, el sistema interpreta la información recibida, identificando las condiciones ambientales actuales, como la presencia de lluvia o valores de temperatura elevados o bajos.

Finalmente, en la etapa de ejecución, el microcontrolador toma decisiones en función de los datos analizados, activando el servomotor para abrir o cerrar la cobertura del sistema. Por ejemplo, si se detecta lluvia, el sistema prioriza esta condición y procede al cierre inmediato del techo para proteger el café.

Este ciclo se repite de manera continua mientras el sistema esté en funcionamiento, permitiendo una respuesta automática y en tiempo real ante variaciones en el entorno, lo que garantiza una mayor eficiencia en el proceso de secado del café.

